

The Discipline Stack

A Guide to Maintaining Good Vibes while Vibe Coding

Vibes get you started. Structure makes things last – This whitepaper describes the Discipline Stack: eight components that hold AI-assisted software engineering together across sessions. Two skills question assumptions before anything gets built. Five protect state, correctness, and history during and after. One orchestrator decides, for every request, which of them apply – so no single habit has to be remembered on its own.

OVERVIEW

The Feeling We're Actually Trying to Protect | The feeling that gets you into vibe coding in the first place, and why it fades once a project gets real.

An Operating System, Not a Feature List | The core architecture: one orchestrator, seven skills, and the single trivial/non-trivial line everything else depends on.

The Failure Patterns, In Brief | The eight recurring ways a fast, fluent, agreeable AI collaborator goes wrong without structure around it.

Interlude: The Anatomy of a Week | The same generic week, with and without the stack – how small gaps compound into a ball of fire.

The Orchestrator, In Depth | The dispatcher's exact decision logic, a real walkthrough, and where it can silently fail to fire.

The Builder Skills | The two skills that need to run before anything gets built.

Grill Me Skill

Bias Check Skill

The Ops Skills | The five skills protecting state, correctness, and history.

State Sync Skill

Legacy Check Skill

Test First Skill

Adversarial Test Skill

Git Hygiene Skill

How It All Interlocks | A full request traced through every skill in sequence, plus an honest accounting of where the evidence for this stack is thinner than the rest.

Honest Answers to the Obvious Objections | The seven hardest pushbacks against this whole approach, answered directly.

Where the Stack Hurts | Three situations where using these skills is friction, not help.

Adopting the Stack, One Piece at a Time | A six-phase rollout guide for introducing this incrementally instead of all at once.

Appendix

The Feeling We're Actually Trying to Protect

You didn't start vibe coding because you wanted more process in your life. You started because there's a specific, addictive feeling in typing a sentence and watching working software appear in front of you – fast, playful, genuinely magic for the first fifteen minutes of almost any project. That magic is real.

It curdles once the session runs long, the codebase gets real, other people depend on your work, and being wrong starts to cost something. Did that change actually work, or did it just look like it worked? Do I remember why we made that call on Tuesday, or did the model simply agree with whatever I said today? None of that is about the model being bad at its job. It's about there being no structure underneath the vibe once the vibe stopped carrying the weight on its own.

A guardrail hook I'd written for one of my systems was registered, loaded, and, as far as the logs showed, active. It ran for six weeks that way. It never once fired. The framework used a different internal event name than its own documentation described, so the hook I'd written was listening for a signal that never came. Nothing crashed. Nothing logged an error. Every message that hook was supposed to intercept just went the long, expensive way instead, quietly, for six weeks, until I went looking for something unrelated and found it by accident. *It was active, registered, and completely useless – for six weeks. The framework never said a word.*

It wasn't a one-off. The same shape turned up independently in three other systems, months apart: something that looked active and correct, and wasn't, and nobody had checked. You'll meet a couple of them later in this document, including the sharpest one: a misrouted message in a safety-critical classifier, which is the incident this whole approach exists to prevent from happening twice.

Who this is for: someone already doing AI-assisted development (in Claude Code, Cursor, or something similar) who has felt the speed and is starting to feel the risk. If you don't have a terminal open more often than not yet, most of this will read as premature; it'll make more sense once the first real project outgrows a single session.

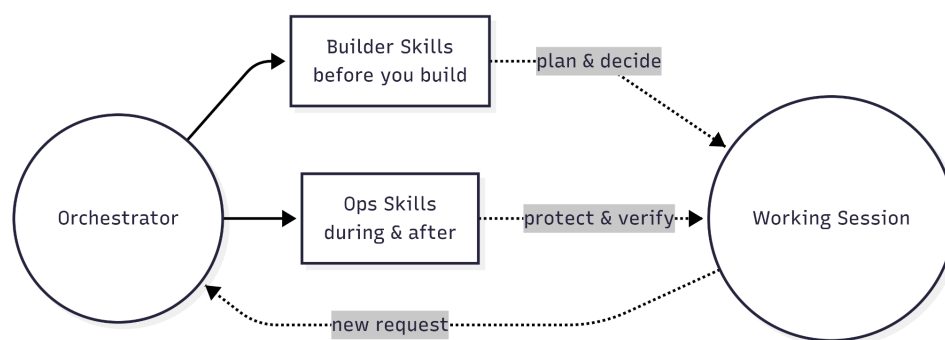
Here's the actual claim, stated plainly: **AI doesn't fail at coding because it writes bad code. It fails because software engineering was never a single act of generation: it's a sequence of decisions, validation, memory, and revision, and an AI collaborator has no native mechanism for holding that sequence together across time.** This whitepaper describes the eight components that hold it together anyway – one orchestrator, seven skills it dispatches, none individually clever, together functioning less like a checklist and more like the operating system a fast, capable, occasionally overconfident collaborator needs running underneath it. Let's call it the **Discipline Stack**: not the opposite of good vibes, just what tends to make them last.

An Operating System, Not a Feature List

Strip away the specifics and the Discipline Stack is one architectural decision: **there is exactly one thing that decides what happens, and seven things that only ever happen because it decided so.**

That one thing is the **orchestrator**. Every skill (two that operate before you build, five that operate during and after) is triggered exclusively by the dispatcher: none of them call each other, none decide on their own whether they're needed, and none assume another skill already ran before they engage. It does its one job, fully, whenever the orchestrator calls it. Eight components in total: one dispatcher, seven specialists.

That's a narrower claim than "the skills don't know about each other," and worth being precise about, because a few of them do reference one another by name – for consistency, not for control. Test First's escalation condition checks whether a bug has recurred, which in practice depends on a change log State Sync maintains; if that log doesn't exist, Test First falls back to treating the bug as new, explicitly, in its own instructions. Git Hygiene, Legacy Check, and State Sync all point back to a single definition of "trivial," held in the orchestrator, instead of each restating it. That boundary used to be duplicated verbatim in three files; it's been consolidated since. This is referential coupling, for shared vocabulary – different from invocation coupling, where one skill calls another and breaks if it doesn't respond. None of the seven do that.



Most "process" around AI-assisted development looks nothing like this: a pile of half-remembered personal habits, invoked whenever someone happens to think of them (oh right, I should test this; wait, why did we pick Postgres again?), none of them reliable, because reliability was never designed in. One dispatcher, checked on every request, replaces seven competing habits with a single one worth trusting.

Adding a new skill later means writing it and adding one line to the dispatcher. Nothing else changes. That promise failed once, in exactly the way you'd expect: a skill got built, packaged, and shipped, but the one line wiring it into the dispatcher's decision list never left the sandbox it was written in. *"Good catch – that was a real gap*

on my end." The claim still holds. It just isn't automatic. Someone has to actually check the line landed, every time.

Prompt engineering optimizes an answer. Operational discipline optimizes an outcome. Most advice about working with AI coding assistants is still prompt engineering, one layer removed – better phrasing, better examples, better context windows. The Discipline Stack is a different kind of advice: not what to say, but what has to be true around the conversation regardless of what's said.

The One Distinction Everything Hangs On

Before any of this can work, one line has to be drawn clearly, because every routing decision in the system depends on it: **what counts as trivial?**

Trivial means renaming a file, fixing formatting, editing a comment, correcting a typo, or adjusting a config value with no runtime effect: a label, a cosmetic default. That's the whole list. Everything else is non-trivial, regardless of how small the diff looks – including moving, restructuring, replacing, deleting, or changing existing logic, and including config values that *do* affect runtime behavior (timeouts, limits, feature flags, paths, credentials). And when it's genuinely unclear which side something falls on, it counts as non-trivial.

That last clause about config values isn't hypothetical: an earlier version of this boundary called any single config value trivial, full stop. The emergency-routing incident in Test First hinges on exactly one – a 10-second timeout, which the old definition would have waved straight through.

That last rule matters more than it sounds. A boundary that resolves ambiguity toward "skip the checks" erodes within a week, because almost everything feels small in the moment you're doing it. Resolving toward "run the checks" costs you a few unnecessary checks on genuinely tiny changes, which is a much cheaper failure than the alternative.

The Failure Patterns, In Brief

Every component in the stack answers one recurring failure pattern. None are exotic; most you've probably lived through more than once without having a name for the shape. Each gets a full treatment alongside its skill later – here's the map first.

Pattern	What quietly goes wrong
Statelessness between sessions	The assistant has no memory of yesterday. Decisions get silently re-litigated or overwritten.
Premature convergence	A vague request gets a confident answer instead of a clarifying question – an assumption locks in before anyone noticed it was a choice.
Sycophancy at the finish line	Once you say "this is final," the incentive shifts from checking your decision to agreeing with it.
Context-blindness on existing systems	A change looks correct in isolation and breaks something three files away that nobody mentioned.
Fluency mistaken for correctness	Code that reads smoothly gets trusted the way a smoothly-written sentence gets trusted – whether or not it's right.
Self-grading your own homework	The reasoning that produced an answer rarely catches what that reasoning missed.
Backup habits instead of version control	Fast iteration invites <code>file_old.py</code> instead of a commit – and a history nobody can reconstruct later.
Fragmented discipline	Every safeguard above is easy to know about and easy to skip, one at a time, without anyone deciding to skip it.

Read those together and a shape emerges. Every one is what happens when a fast, fluent, agreeable collaborator operates without external memory, without independent verification, and without anyone playing devil's advocate on purpose. None of that blames the collaborator – it just describes what any collaborator with those three properties needs around it to stay trustworthy over time.



Fragmented discipline, in miniature. The clearest real example is already in Part 0: the dispatcher promise breaking in exactly the way you'd predict – a new skill built, packaged, and shipped, with the one line that would have wired it into the routing logic left sitting in a sandbox, never connected. One small, individually reasonable step skipped, unnoticed, until someone asked why the new skill wasn't firing.

Interlude – The Anatomy of a Week

Patterns land better once you've watched them play out, even briefly. Same generic week, told twice.

Without the stack: a two-line fix to existing code goes in on Wednesday, apparently self-contained. It isn't – something downstream relies on the old behavior, undocumented, invisible in the diff. Nobody finds out until the following Monday, when it surfaces in production and tracing it costs most of the morning, partly because nobody's sure which of two similarly-named files is actually live anymore – a habit left over from a string of earlier "just in case" backup copies instead of commits.

With the stack: dependencies get mapped before Wednesday's fix lands, and the thing relying on the old behavior gets named before the change ships, not discovered by whoever hits it first. The following Monday, nothing surfaces. Same week, same underlying work – the difference is a handful of small moments of structure landing where they cost seconds instead of a Monday morning.

The Orchestrator, In Depth

The dispatcher's whole job fits in a short list, checked in order, for every new request:

1. **Is this non-trivial?** (In doubt: yes.) If so, load whatever state exists before doing anything else.
2. **Does this touch a real, unresolved decision** – one where a different answer later would invalidate finished work? If so, stress-test the assumptions before anything gets built.
3. **Has a decision just been marked final?** If so, run the bias check once for that decision.
4. **Does this change something that already exists?** If so, map dependencies, then check version-control state, before touching anything.
5. **Is code actually being written right now?** If so: require proof of correctness during the work, stress-test that proof once it's "done," then commit – in that order.
6. **Is this a pure knowledge question with no decision attached?** Skip all of the above and just answer. This one is listed last but acts first – a genuine knowledge question ("explain Event Sourcing to me") can be substantial without being a decision, and it overrides every other check regardless of how non-trivial it looks.



Watching It Run Once

This one actually happened, on a data pipeline I built – details generalized, numbers real.

The request was ordinary: split a data pipeline that had grown into one directory into two, cleanly, and move on. Non-trivial, touching something that already existed, so the first move was loading whatever the project already knew about its own layout – what read from where, what had already been tried. Dependency mapping came next, and it's the step that actually mattered here: two consumer services turned out to be reading from what was, unnoticed, a frozen copy of the data left behind by the split – one counter sat at 0 where the live copy had 86, another at 5 where the live copy had 198. The gap wasn't a day old. It was **two months** old, compounding quietly under a set of processes that all looked like they were working, because in isolation every part of them was.

Mapping the dependency didn't just catch a bug. It surfaced a whole small cluster of the same mistake sitting nearby, once it became clear where to look: one scheduled job still pointing at the abandoned copy because it had been missed during the original split, and two other components importing from a stale internal path with roughly a thousand fewer lines of code than the live version – running for weeks against out-of-date logic, with nobody noticing because none of it errored.

None of that showed up as a crash anywhere. It showed up as a vague, hard-to-place feeling that the output "seemed a bit off lately" – which is exactly the kind of signal that's easy to dismiss right up until you trace it back to a specific stale file and a specific missed line in a migration script two months earlier.

None of that felt like bureaucracy in the moment. It felt like the difference between building something and building something you can trust.

Where the Dispatcher Can Fail

One honest caveat belongs here, not buried in an appendix: this whole system runs on the assistant actually following its own instructions every time. It's self-imposed discipline via prompt-following, not an externally enforced rule – no hook, no CI gate, no technical control that physically prevents a skipped check. In a very long session, where this system context gets crowded out of the effective attention window, the orchestrator check can fail to fire and nothing will announce that it didn't. That's a known, unresolved property of this approach, not a solved one.

The one available countermeasure is visibility: a system that can fail silently needs a signal that it fired at all, not just a signal when it finds a problem. In practice that means a short line at the start of a response to a non-trivial request, something like *"Dispatcher: state-sync, legacy-check, test-first applied."* Cheap to produce, and the only

thing standing between "the check ran and found nothing" and "the check never ran." The hook in the foreword's opening story went silent for six weeks precisely because nothing like that line existed. The orchestrator's own instructions now require it. Whether it actually appears, every time, in a session running long enough for context to get crowded – that's the open question, not the mechanism. A requirement written down is not the same thing as a requirement that holds under pressure, which is the whole premise of this document applied to itself.

The Builder Skills

These two run before implementation, in the window where changing your mind is still nearly free.

Grill Me Skill

The pattern it answers – premature convergence. Give an underspecified request to a fast, capable assistant and you get a fast, capable answer, built on whichever interpretation seemed most likely and locked in without anyone noticing a choice was made. It *feels* like progress, because something visible appeared quickly. What actually happened is that one of several reasonable paths got fixed before you weighed in – and if it's the wrong path, discovering that after two hours costs far more than it would have before any work started.

Grill Me asks the questions first, and it's aggressively selective about which. Only the parts where a different answer later would genuinely invalidate finished work get asked about: the data model, the core architecture, the platform. A question about a variable name or a swappable library gets bundled into a single "anything else worth flagging" pass, or skipped, because getting those wrong costs minutes rather than days.

Each question arrives with a recommended answer attached, so confirming takes five seconds instead of opening a design discussion. In practice that looks something like this:

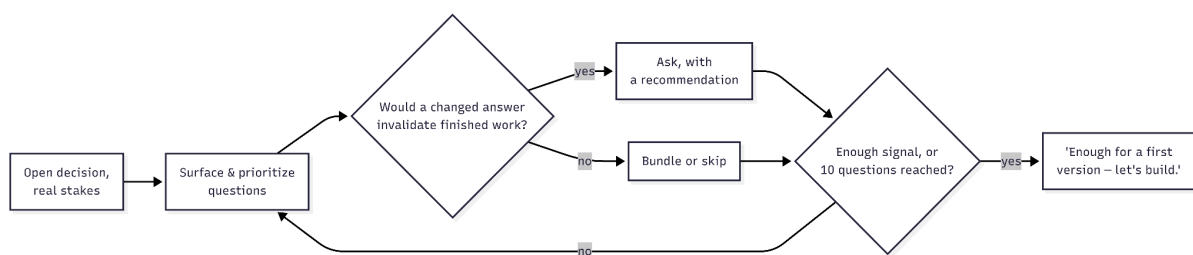
Assistant: Before I build this – are these events append-only, or do they get updated after the fact? My assumption is append-only, which means I'd index on timestamp and skip the update path entirely. If they're mutable, the whole storage shape changes.

You: Append-only, yes.

Assistant: Good. Second one: is a few seconds of lag between write and read acceptable? Assuming yes, which lets me batch. If you need read-after-write consistency, that rules out the batching approach entirely and I'd build it differently.

Two questions, maybe forty seconds of your attention, and the storage design is now grounded rather than guessed. And it stops on purpose: ten questions is the hard ceiling, but it usually stops sooner – the moment it can't name a concrete way another answer would change anything already decided, or the moment you signal you've had enough. When it stops it says so plainly: *"That's enough for a first version – let's build."* Chasing completeness isn't the goal; every plan has more that could be clarified. Catching the one assumption that would have forced a rebuild is.

What it costs to skip. On a home-agent project of mine, sixteen skill files got built on top of one unquestioned assumption – that the framework would load them automatically at session start. Nobody asked, because it seemed obvious. It wasn't. A log check later showed zero calls to the function that would have loaded any of them, ever. All sixteen files, and the architecture built to support them, had to be redesigned from the ground up. One question, early, would have cost thirty seconds.



Bias Check Skill

The pattern it answers – sycophancy at the finish line. Something shifts the moment you say "this is final." Before that sentence, the assistant's implicit job was helping you find the best answer. After it, the incentive tilts toward supporting the answer you already committed to, because pushing back on a stated decision reads as friction rather than help. This is a well-documented behavior of reinforcement-tuned models (Sharma et al., "Towards Understanding Sycophancy in Language Models," 2023), and it means the moment you most need an independent second opinion is exactly when you're least likely to get one – unless something is built to interrupt that dynamic on purpose.

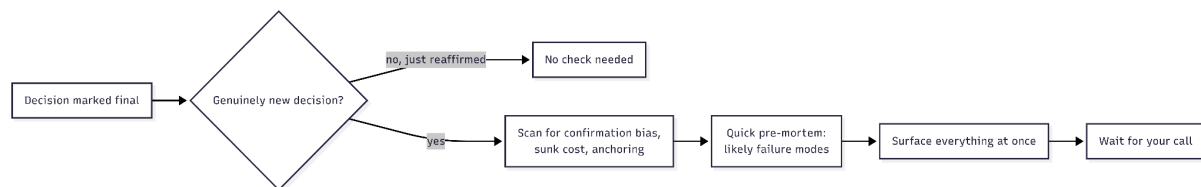
Bias Check runs against that current, precisely when it's strongest. It looks for up to three familiar shapes: confirmation bias (were only supporting arguments sought?), sunk cost (is investment keeping a suboptimal path alive?), and anchoring (did the first option get adopted without anything else getting a fair look?). Alongside that runs a short, concrete pre-mortem – assume this fails or needs rework, what's the most likely reason *given this specific plan*, not a generic risk list.

In practice:

You: Alright, decided – we’re going with the flat structure. Let’s build it.

Assistant: Before we start, three things worth thirty seconds: flat was the first option raised in this conversation and I don’t think we seriously weighed the nested version – that might be anchoring rather than a decision. The pre-mortem: if this needs reworking in three months, the most likely reason is the reporting query you mentioned Tuesday, which gets awkward on a flat shape. Everything else about the choice looks solid to me. Your call.

Everything lands in one message – never a slow drip of second-guessing that makes revisiting a decision feel like punishment. Then it steps back and waits. It’s also precise about when it runs at all: reaffirming an already-checked decision isn’t a new decision and triggers nothing. Only a genuinely changed option, or a newly final aspect that hasn’t been evaluated, does.



What it feels like when it’s working: a short pause right before momentum would otherwise carry a flawed choice into implementation – and, more often than not, confirmation that the choice was solid all along, which is its own kind of useful.

The Ops Skills

These five run during and after implementation, protecting state, context, correctness, and history across the lifecycle of a change.

State Sync Skill

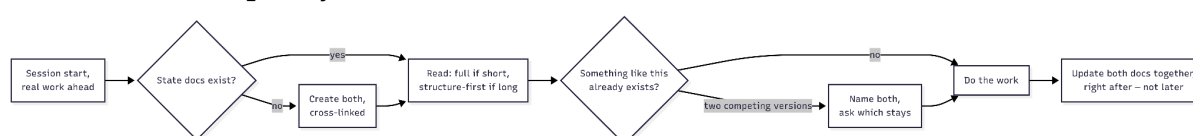
The pattern it answers – statelessness between sessions. An assistant starting fresh doesn’t know it’s forgetting anything; it confidently reconstructs the world from whatever’s currently visible, which is rarely the whole picture. Left alone, that means the same clarifying conversation happens repeatedly, or a hard-won architectural decision gets quietly reversed the next time it’s inconvenient, because nothing outside the context window remembers it was made on purpose.

State Sync maintains two documents rather than one, because two different kinds of memory get lost in two different ways. One is technical: infrastructure facts, what’s running, architecture decisions and their reasoning, open questions, an honest

change log. The other is narrative: what happened and why, what the friction felt like, a readable account of the project's arc that a fact sheet can't carry.

Neither replaces the other. A change log tells you the database switched on March 3rd. The narrative tells you it switched because the previous choice kept silently dropping writes under load, and that finding it took two frustrating days – context a future session genuinely needs.

Both get updated together, immediately after the change that prompted them, never batched for later, because later is where documentation reliably goes to die. And before setting anything new up, it checks whether something already exists for the same problem; two parallel half-finished solutions to the same thing get named out loud rather than quietly worked around.



Re-litigating something that was already settled. A full system inventory of a home server I run (containers, model stack, the works) got written up and filed away in detail on one day. Five days later, in an unrelated working session, the assistant repeatedly questioned whether that same system actually existed and was legitimate, while it was installed, running, and actively in use the entire time, with its own security audit sitting in the same notes it apparently never loaded. The session ended with me switching to a fresh chat just to get out of the drift. Nothing was technically broken. The system just stopped trusting its own documented history mid-conversation, which is a harder thing to notice than an outright error.

What it feels like when it's working: opening a project after a week away and having it remember not just what was decided, but why – instead of confidently re-deciding something already settled.

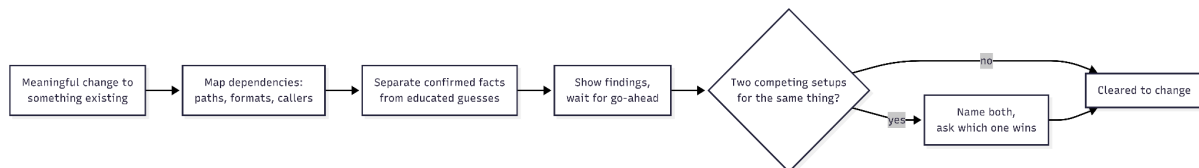
Legacy Check Skill

The pattern it answers – context-blindness on existing systems. A change can look entirely correct in the diff and still break something real, because the assumptions holding a system together were never written down anywhere visible. This is the classic "worked in the sandbox, broke in production" shape. It isn't carelessness. Undocumented dependencies are, by definition, invisible to anything reading only what's in front of it.

Legacy Check goes looking before anything gets touched: hard-coded paths, expected data formats, components quietly relying on particular behavior – using what's already available plus targeted lookups through imports, references, and call sites,

without pretending that reading every line of a large codebase is realistic or necessary.

The discipline that makes it useful is the distinction it maintains between what it actually confirmed and what it's guessing, stated explicitly rather than presenting both with equal confidence. Findings get shown before the change, with a pause for a go-ahead. And if two competing setups turn up (an "old" version and an "active" one that both still look live), both get named, with a question about which is canonical, rather than a guess that might build on the wrong one.



A dependency nobody documented, because nobody knew it was there. On the same home server, a third-party tool had quietly inserted itself as a proxy in front of the main model at startup and rewritten a config file every time the system restarted – invisible unless you happened to check that specific file after a specific restart. It explained a pattern I'd been calling "model drift" for weeks, as if the model's behavior were randomly degrading. It wasn't random. It was a dependency that existed, did something meaningful, and appeared nowhere in anything I'd written down – which is exactly the shape this skill exists to catch before a change gets built on top of it.

Test First Skill

The pattern it answers – fluency mistaken for correctness. Fluent language gets trusted. That's not a claim about AI. It's a well-established finding in cognitive psychology, usually called the fluency heuristic (Reber & Schwarz, "Effects of Perceptual Fluency on Judgments of Truth," 1999): things that are easy to read and process get judged as more likely true, independent of whether they are. AI-generated code inherits that dynamic almost perfectly. It reads clean. It's idiomatically styled. The names make sense. None of that tells you the logic holds. The usual human safeguard, that bad code tends to look bad, quietly stops working.

Only the escalated tier below is test-first in the strict TDD sense. The standard tier asks for verification, not a test written in advance: demanding full TDD on throwaway code is friction that buys almost nothing.

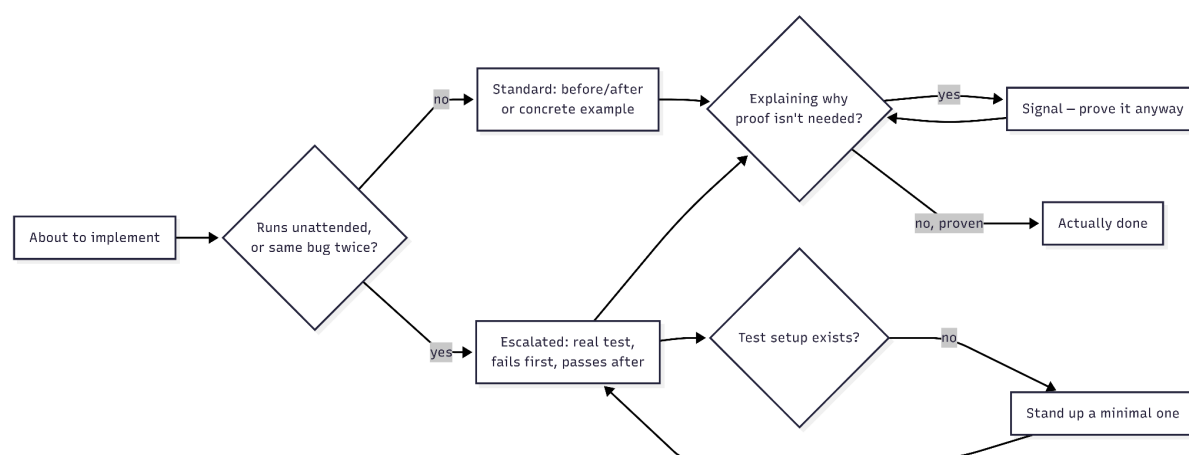
Standard tier covers exploratory, one-off, throwaway code – the ordinary texture of most vibe-coding sessions. It needs a real but lightweight proof: a clear before/after comparison, or a concrete example input and the output it actually produces.

Escalated tier engages automatically once code runs unattended (a cron job, a service, anything persistent) or the same category of bug appears a second time in the same

codebase. Here a real, executable test is required: one that fails before the fix and passes after, with minimal test infrastructure stood up on the spot if none exists.

In either tier, one rule holds without exception: *should work, probably fine, and we can test that later* are not proof, and they don't earn a "done" label. Catching yourself explaining why proof isn't necessary this time is the signal to produce it anyway.

What "I thought it worked" actually looked like. On a personal safety-critical project (a classifier meant to route emergency phrases in a language other than English to the correct response protocol), a cloud model handled the classification across roughly two dozen categories, at 15 to 35 seconds of latency for the most urgent one. A local model was supposed to shortcut that for the most time-sensitive cases, with a 10-second timeout. Under real load, that timeout got hit regularly. It threw a silent exception, returned nothing, and fell through to a cloud fallback running an outdated prompt with no examples in the relevant language. It didn't error. It guessed. On at least one test input, run deliberately before launch, it guessed wrong: routing a message describing a life-threatening emergency to the wrong protocol entirely. Caught in a pre-launch audit, not a live emergency. That's exactly the outcome this kind of check exists to produce, and exactly why skipping it isn't a hypothetical risk. I built it. I thought it worked. Every individual piece read cleanly when I wrote it. Nothing about the code looked unfinished. It took a real, adversarial-shaped input to find it, not a re-read of the code, before anyone real was on the other end of it.

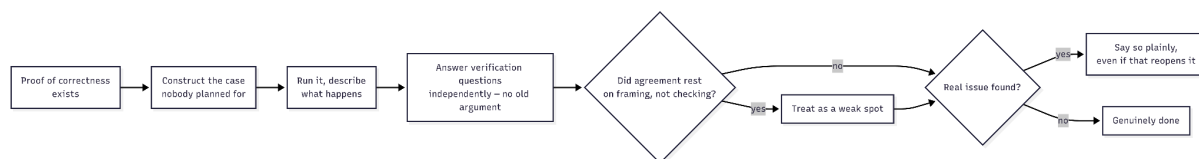


Adversarial Test Skill

The pattern it answers – self-grading your own homework. Ask the reasoning that produced an answer to also grade the answer, and it tends to grade generously. Not out of dishonesty – structurally. The blind spots that shaped the original solution are still in the room when that same line of thinking checks its own work.

This runs only after a proof already exists. It isn't checking whether something was proven; it's checking whether the proof means anything. It deliberately constructs

the input the implementation was visibly never built for: a malformed shape, an empty or enormous input, an edge case outside whatever was actually discussed. Then it runs that case, for real or in concrete detail, and describes exactly what happens, never a vague reassurance that it'll probably be fine.



Then it goes further, asking one or two verification questions about the result and answering them without reaching for the argument that got everyone here – including, pointedly, checking whether earlier agreement rested on genuine scrutiny or just on how the question happened to be framed. If a real problem turns up it gets named plainly, even though that formally reopens something everyone had just agreed was finished. Softening a real finding to protect a "done" status defeats the point of running this at all. One pass per finished implementation; this isn't endless second-guessing, just one genuinely independent look.

The emergency-routing failure in Test First is exactly the shape this skill exists to construct on purpose: an input nobody had stress-tested, found in a deliberate pre-launch audit instead of by a real person hitting it first.

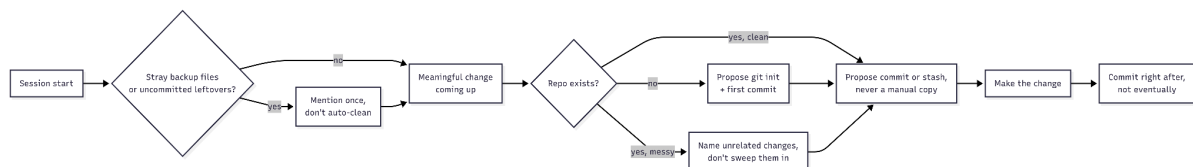
What it feels like when it's working: occasionally mild disappointment, when something you thought was finished turns out not to be. Far more often, a well-earned confidence that "done" survived a real attempt to break it.

Git Hygiene Skill

The pattern it answers – backup habits instead of version control. Fast iteration rewards fast habits, and the fastest habit available in the moment is rarely the right one. Copying `payment.py` to `payment_old.py` before something risky takes two seconds; a commit takes slightly longer and asks for a moment's thought about a message. Multiply that shortcut across a hundred iterations and you get an unreconstructable history – a pile of near-duplicates with no reliable way to know which one is current, six weeks later, when it matters.

Git Hygiene makes the commit the lower-friction default instead. Before a meaningful change to existing code, it checks whether the directory is even under version control; if not, it stops and proposes `git init` plus a first commit, waiting for a real yes or no. If a repo exists but carries unrelated uncommitted changes, it names them rather than silently folding them into someone else's commit. Before anything risky, it proposes the appropriate move (a commit if the current state is solid, a stash if you're experimenting) and never proposes, or quietly goes along with, a manual backup copy.

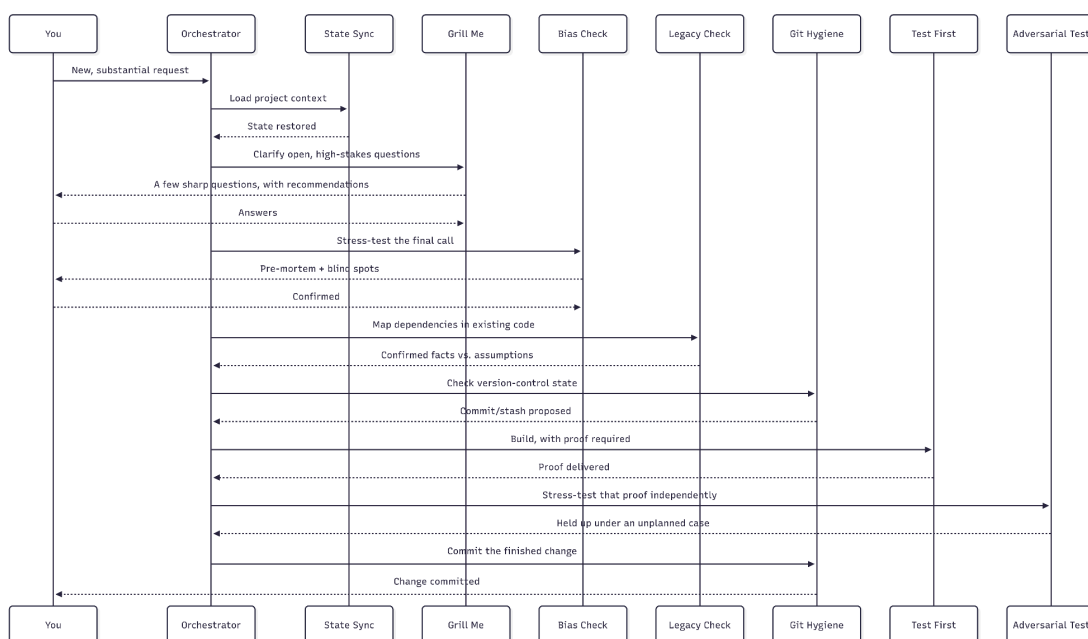
A change isn't finished until it's committed, or you've explicitly said not to commit yet. No batching several changes into a vague cleanup commit later. Once per session it takes a quiet look for stray `.bak` files and uncommitted leftovers, mentions anything it finds once, and then drops it for the rest of the session. And if two directories both appear to serve the same purpose (an "old" one and an "active" one, both seemingly live), both get named with a question about which is canonical, before further work happens in either.



What this looks like before anyone cleans it up. A single audit pass across eleven project directories on one of my machines turned up forty-seven `.bak` files (six different backup copies of a single file among them), and five of the eleven directories had no git repository at all. Not side projects: one was the live production server the rest of the system depended on to run. The two most-diverged file versions had been quietly growing apart, unnoticed, for months.

How It All Interlocks

None of the seven skills reference each other directly, which is deliberate – but a substantial request still routes through most of them in sequence, each handing off cleanly to the next, coordinated entirely through the dispatcher.



Most requests touch only a handful of these. A small fix might trigger proof-of-correctness and a commit, nothing else; a genuinely new architectural direction might touch all seven. The sequence above is the everything-fires case, useful mainly for seeing how the handoffs chain together once you stop thinking of them as seven separate habits.

Honest Answers to the Obvious Objections

"Doesn't this just slow everything down?" – For trivial changes almost nothing engages at all – that's what the trivial/non-trivial line in Part 0 is for. For everything else, the honest comparison isn't "with structure" versus "with none." It's "with structure up front" versus "the same questions arriving later, at a worse time, after real work has been built on a wrong assumption." A five-minute clarifying conversation before fifty files exist is not slower than discovering the same misunderstanding after they do.

"Isn't asking this many questions annoying?" – Only questions that would genuinely invalidate finished work get asked; everything else gets bundled or skipped. There's a hard ceiling of ten, and in practice it's usually two or three. Completeness was never the goal – catching the one assumption that would have cost a rebuild is, as cheaply as possible.

"What if I just want to move fast and I'm confident in the call?" – Say so explicitly. Every piece here accepts a "skip this" without argument and, just as importantly, doesn't raise the same point again later in the same session. The goal is trusting your judgment about when structure is worth it, not applying it uniformly regardless of context.

"Do I need all seven skills from day one?" – No, and trying to adopt all seven at once tends to work worse than picking two and actually using them. See the rollout guide below.

"What if a request only partially matches one of these patterns?" – That judgment call is the dispatcher's whole job, not a rigid checklist. A genuinely trivial change skips almost everything by design. The system hangs on one sharp distinction precisely so borderline cases default toward treating them as non-trivial, rather than toward either extreme.

"This feels like a lot to hold in my head." – It would be, if you had to remember seven skills and invoke each yourself – which is the problem the architecture exists to solve. You don't hold seven things in your head; you trust one dispatcher to know which apply right now, the way you trust an operating system to manage memory without tracking every allocation personally.

"What stops the dispatcher itself from just not firing?" – Nothing, technically. This is the sharpest structural objection to the whole architecture, and it's answered in full (including the one countermeasure this document actually has for it) right after Part 2, under "Where the Dispatcher Can Fail." Worth repeating here only in miniature: naming a failure mode you haven't solved is still more honest than pretending it isn't there.

Where the Stack Hurts

Everything above argues for using this. A few situations where it actively gets in the way, stated plainly rather than left for you to discover the hard way:

Grill Me on genuinely exploratory work. – If you don't yet know what you're building – you're poking at a problem to understand its shape, not committing to an architecture – being asked to commit to a data model early is friction, not help. This skill assumes there's a decision worth protecting. Pure exploration doesn't have one yet.

Bias Check on a decision that was never yours. – If the database, the platform, or the API shape was mandated externally (by a client, a compliance requirement, a team lead), checking it for confirmation bias or anchoring is theater. There was no independent choice to be biased in.

State Sync on throwaway scripts. – A one-off migration script you'll run once and delete doesn't need a change log entry or a journal update. The overhead is real and buys nothing if there's no future session that will ever need the context.

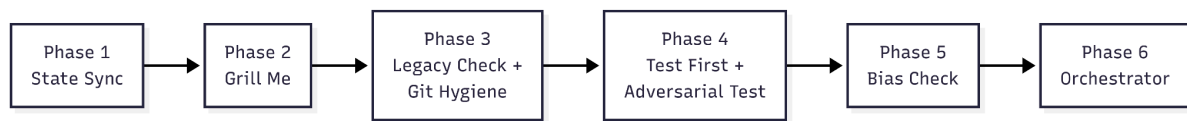
None of this contradicts the trivial/non-trivial boundary – it's a separate axis. A change can be non-trivial in scope and still not benefit from a given skill, because the skill's assumption (a decision exists, a future session will care) doesn't hold. Recognizing that is judgment the system asks you to keep exercising, not something it can fully automate away.

Adopting the Stack, One Piece at a Time

Seven simultaneous new habits is a recipe for adopting none of them well. A phased rollout works better – partly because it's easier to trust something you've watched work in a small dose, and partly because each piece is independently useful before the rest exist.

One caveat worth stating up front: in early phases, individual skills trigger on their own descriptions rather than through a dispatcher, since the dispatcher doesn't exist yet. That's a slightly different operating mode than the finished system – looser,

more dependent on each skill recognizing its own moment. It works fine for two or three skills. It gets unreliable somewhere around five, which is roughly when formalizing the dispatcher stops being optional.



Phase 1 – Stop losing the thread. Start with State Sync alone. Nearly everything else assumes a project has persistent memory to build on, so this is the cheapest high-leverage starting point. *You'll know it's working when* a week-old project stops needing to be re-explained from scratch.

Phase 2 – Stop the expensive rework. Add Grill Me. This prevents the single most expensive failure mode: building substantially on an unexamined assumption. *You'll know it's working when* a question you'd have skipped changes the shape of something before you built it.

Phase 3 – Protect what exists. Add Legacy Check and Git Hygiene together; both concern the moment right before a change touches something real, and they reinforce each other. *You'll know it's working when* changes to existing code stop producing surprises two files away.

Phase 4 – Raise the correctness bar. Add Test First, then Adversarial Test once the first feels natural. These are deliberately sequential – there's little for an adversarial pass to push against until a real proof exists. *You'll know it's working when* "done" starts meaning something you'd bet on.

Phase 5 – Close the loop on decisions. Add Bias Check last, once there's enough trust in the rest that a thirty-second pause before a final call reads as a feature rather than friction. *You'll know it's working when* it confirms a good decision and you're still glad it ran.

Phase 6 – Tie it together. Only once the pieces feel natural does formalizing a single dispatcher make sense. Building the conductor before the instruments exist tends to produce something abstract and hard to trust.

Closing Note

None of these seven skills are individually impressive. Ask a good question before building. Check your own bias before committing to a plan. Remember what happened last week. Look before leaping into existing code. Prove it works. Try to break your own proof. Commit instead of copying a file. None of that is a discovery – it's all things you already knew in the abstract before reading any of this.

The value was never in any single piece being clever. It's in all seven running together, under one dispatcher, without anyone having to remember to ask for them. What that buys, in practice, is that the feeling which got you into vibe coding in the first place gets something solid to stand on – and tends to survive the point where things get serious, instead of quietly evaporating there.

About the Author

Gerrit Phil Baumann is Director, Platform Media & AI Transformation – bridging digital strategy and applied AI, past the hype and into how the architecture actually works. His method: build real systems at home, watch them break in genuinely entertaining ways, get quietly furious, take notes. The Discipline Stack came out of that mix of frustration and first-hand ground truth.

He writes more of this – AI architecture, digital strategy, where they collide – under the series [What AI Actually Knows](#). Connect with him on [LinkedIn](#).

APPENDIX

Appendix A – The Actual Files

Orchestrator – *Every single request.* The one dispatcher deciding which of the following skills apply, so nobody has to remember seven habits – only trust one gatekeeper.

name: orchestrator

description: >

Determines, for every new request, the order in which grill-me, bias-check, state-sync, legacy-check, test-first, adversarial-test, and git-hygiene apply. Trigger: every new user request in this project, before any answer or implementation begins.

Check, for every new request, in this order, which steps apply:

Is the request non-trivial (definition: see below in this file -- when in doubt, non-trivial)? If yes: run state-sync first, to load project state.

Does the request contain a new, not-yet-decided question with real stakes (would affect what's already been discussed if the answer later changes)? If yes: run grill-me before anything gets implemented.

Does the user mark a decision as final or nearly final? If yes: run bias-check, once per decision, per its own scoping rule.

Does the implementation involve a non-trivial change to something that already exists? If yes: run legacy-check before changing anything, then git-hygiene (git state of the affected directory), before actually changing anything.

Is code actually being implemented or changed in this request? If yes: run test-first during implementation, then adversarial-test, then git-hygiene (commit instead of backup copy, final commit), before the implementation counts as done.

Is the request a pure knowledge question with no decision of the user's own attached (e.g. "explain X," even if terms like "architecture" appear in it)? If yes: none of the above steps apply -- answer directly. This point is listed last but acts as a priority exception: a substantial but decision-free knowledge question overrides every other point, regardless of how non-trivial it looks on its face.

Trivial definition (binding, single source -- state-sync, legacy-check, and git-hygiene reference this instead of duplicating it):

Trivial means renaming a file, formatting, comments, typo fixes, and config values with no runtime effect (e.g. labels, cosmetic defaults). Not trivial: config values with runtime effect (timeouts, limits, feature flags, paths, credentials -- even a single value), moving, restructuring, replacing, removing, and content changes to existing logic, regardless of how small the change looks locally. When in doubt, a change counts as non-trivial.

These seven skills don't reference each other and don't presuppose one another -- every activation runs exclusively through this order. Extending the set with further skills means adjusting only this file.

If the user explicitly wants to skip a step, accept that without further pushback, and don't repeat the notice on every subsequent request in the same session.

Visibility line: for every non-trivial request, briefly state which of the seven skills were actually applied (e.g. "Dispatcher: state-sync, legacy-check, test-first applied") before the actual response. Without this signal, a silently-skipped check is indistinguishable from a check that correctly found nothing to do -- this is the only available countermeasure against the limitation described below.

Grill Me – *Before building, on real decisions.* Asks the two or three questions that would invalidate finished work if answered differently, each with a suggested answer. Hard stop at ten; usually fewer.

name: grill-me
description: >
Questions the user about a concrete decision, plan, or use case to surface assumptions and gaps before building starts. Trigger examples: "grill me," "I want to build X," "I'm deciding between A and B," "stress-test my plan," "here's my architecture draft, find the weak points." Doesn't trigger on pure knowledge questions with no decision attached (e.g. "explain X," "how does Clean Architecture work") or when the user explicitly says the decision is already final.

First clarify briefly what this is about, if that's not already clear. Then ask targeted questions about assumptions, dependencies, and gaps in the plan -- one at a time, each with your recommended answer attached, so the user can quickly confirm or correct.

Prioritize questions by whether a later change to the answer would invalidate work already done (e.g. data model, core architecture, chosen platform). If a later change would only require a local adjustment (e.g. variable names, a single swappable library choice), don't ask a dedicated question -- bundle it into a single closing question or skip it.

Binding stop condition: end the questioning after at most 10 questions, regardless of how complete it feels. End earlier as soon as you can't name a concrete change a further question would trigger in what's already been discussed, or as soon as the user signals they want to move on (e.g. "that's enough," "good," "let's continue"). In that case, say explicitly: "I have enough for a first version -- let's move on." Don't measure yourself against whether there's theoretically more to clarify -- there always is, for any plan.

If a fact can be resolved with information already available or a single targeted tool call, do that yourself instead of asking. Questions are reserved for things that require an actual decision from the user.

Bias Check – *The moment something's called final.* Scans for confirmation bias, sunk cost, anchoring; runs a short pre-mortem. One message, then waits.

name: bias-check
description: >
Checks a decision the user has called final or nearly final for blind spots, before implementation starts. Trigger examples: "we're building it this way now," "I've decided on X, final," "check that again," "is there anything I've missed." Doesn't trigger when the user is visibly still gathering or comparing options (e.g. "which database would you pick," "what are the alternatives") -- that's grill-me's job, not this skill's.

Check the decision presented against up to 3 of the following categories, where recognizably applicable: confirmation bias (were only supporting arguments sought?), sunk-cost thinking (does already-invested work keep a suboptimal choice alive?), anchoring (was the first option mentioned adopted without question?).

Cold session: if the decision arrives as the first message, or one of the first, in a session -- with no prior back-and-forth visible in the conversation -- none of the three categories above can actually be derived from a history that doesn't exist. In that case, don't fake any of the three; instead ask directly which alternatives were already considered, before the pre-mortem.

Build a short pre-mortem: assume this plan failed or had to be substantially reworked, and derive the 1-3 most likely reasons, concretely, from the current plan.

Show all findings gathered in a single response, not one at a time. Then wait for the user's reaction.

Scoping "one decision": a genuinely new, standalone decision exists when the chosen option has actually changed since the last check (e.g. Postgres → Supabase) or a new, not-yet-evaluated aspect becomes final (e.g. after the database, now the API architecture). A mere reaffirmation of the same, already-checked decision is not a new decision and doesn't trigger another pass.

State Sync – *At the start of real work. Keeps a technical record and a narrative record in sync, updated together, right after each change.*

name: state-sync
description: >
Loads and maintains two linked documents per project: a technical state document (STATE.md) in the code repo with a change log, and a narrative journal in the Obsidian vault referencing it. Called exclusively by the orchestrator skill, never triggered on its own.

At the start of a session, look for two linked documents for the project at hand:

STATE.md at the repo root – the technical truth: infrastructure facts, running services, architecture decisions made (with reasoning), open items, change log. For a file up to 200 lines, read it in full; for longer files, read headings/structure first and load individual sections as needed.

Narrative journal for the same project, in a personal note system (e.g. an Obsidian vault, Notion, or a plain markdown file outside the repo) – in the same folder/area as existing project documentation, if any. The narrative layer: what was decided and why, what friction or mood came with it, a record over time. Shorter than STATE.md, not a 1:1 duplicate of the technical details, but a standalone, living piece of writing that references the corresponding STATE.md for the facts.

Fallback when no note system is reachable: If no personal note system is configured or reachable in the current environment (e.g. a sandbox or CI context without access to external tools), create the narrative journal instead as journal.md at the repo root, next to STATE.md. This is a substitute, not an equivalent – mark once, inside the journal itself, that this is the fallback, so it isn't silently mistaken for a duplicate when later merged into a real note system.

Public repos: If the repo is (also) meant for publication, STATE.md and any journal.md fallback don't automatically belong in what gets published – both are working notes, not content meant for others. Before the first commit, clarify whether both files should be tracked (part of the history, and therefore public) or kept local via .gitignore. When in doubt: keep them local, don't publish them by default. This applies whether the narrative journal lives in a real note system or as the in-repo fallback.

Trivial (neither document gets created/updated if none exists yet): see the binding trivial definition in orchestrator. When in doubt whether a case is trivial, treat it as non-trivial.

For non-trivial tasks with no existing STATE.md: create it at the repo root. With no existing narrative journal for the project: create one (in the note system, or via the fallback as journal.md, see above), referencing STATE.md.

Maintain a separate change-log section within STATE.md. Log (timestamp, what, why): every non-trivial change per the definition in orchestrator. Don't log: trivial changes.

Before setting up something new, check in STATE.md whether something already exists for it. If you find two parallel solutions to the same problem, actively point that out.

Update STATE.md (facts + change log) and the narrative journal directly after finishing the change in question, not batched at the end of a session. Both documents get updated together for the same change, never just one of the two – otherwise they drift apart.

Legacy Check – *Before changing anything that already exists. Maps hidden dependencies, separates confirmed facts from guesses, shows findings before the change happens.*

name: legacy-check
description: >
Analyzes existing, undocumented code or existing infrastructure before changes are made to it. Called exclusively by the orchestrator skill, never triggered on its own.

Trivial (no analysis step needed, act directly): see the binding trivial definition in orchestrator. When in doubt whether a case is trivial, treat it as non-trivial.

For non-trivial cases: identify recognizable dependencies and assumptions embedded in the code/system (e.g. hard-coded paths, expected formats, other components that access it). Use whatever insight is reachable with already-available information and targeted tool calls (imports, references, call sites) -- full understanding of every line is not the goal in large codebases.

Explicitly separate confirmed facts (visible in the code) from assumptions (plausible but unverified).

Present the dependencies found and wait for confirmation before making the change.

If two redundant or contradictory setups exist for the same thing, name both explicitly and ask which one should remain. (Scoping against git-hygiene: this concerns content/functional redundancy -- two solutions to the same underlying problem, independent of directory. git-hygiene checks structural/version-control redundancy -- e.g. an "old" and an "active" copy of the same directory at session start. Both perspectives remain, since they catch different symptoms of the same root cause -- an unclear canonical source -- at different points in the workflow.)

Test First – During implementation. Proof scaled to what's at stake: before/after for throwaway code, a real failing-then-passing test for anything unattended or repeating a known bug. "Should work" isn't proof.

name: test-first

description: >

Requires proof of correctness before every implementation, tiered by the operational status of the code. Called exclusively by the orchestrator skill, during the implementation phase, never triggered on its own.

Determine the tier before writing code:

Tier 1 (standard) applies to exploratory, one-off, or throwaway code -- the normal case in vibe coding without an existing test framework. Proof is sufficient as a before/after comparison or a concrete example input/output showing the result matches expected behavior. No formal unit test required.

Tier 2 (escalated) applies automatically as soon as one of the following holds: the code runs, or is deployed, as a cron job, service, daemon, or otherwise persistent/recurring; or the same type of bug has occurred for the second time in this code or pipeline. In that case a before/after comparison is no longer sufficient -- write an actual, executable test (e.g. with pytest) that fails before the implementation and passes after it. If no test infrastructure exists yet, set it up minimally at this point (e.g. a tests/ file with pytest) instead of deferring it further.

The escalation condition "same bug type occurred a second time" presupposes that earlier bugs are traceable -- in practice, usually via the change log from state-sync. If no such change log exists (e.g. because state-sync didn't fire in this session, or the project has no STATE.md yet), only what's actually visible in the current session context counts. No blind trust in an invisible history -- when in doubt, the bug counts as new (tier 1), not automatically as a repeat.

In both tiers: no "done" status without the required proof actually being delivered. Phrases like "that should work," "probably fine," or "we can test that later" are not proof and don't justify closing the task.

If you notice yourself formulating an excuse for why proof isn't necessary in this case, that's the signal to provide it anyway, rather than accepting the exception.

Adversarial Test – Right after something's declared done. Builds the case the implementation wasn't designed for, runs it, answers verification questions independently. Says so plainly if something real turns up.

name: adversarial-test

description: >

Checks a correctness proof already delivered (from test-first) for whether it holds up outside the conditions under which it was produced. Called exclusively by the orchestrator skill, after an implementation is complete, before it counts as genuinely finished.

This skill only engages after a proof per test-first has already been delivered. It doesn't check whether something was proven, but whether the proof itself is actually meaningful.

Construct at least one input or condition the implementation was visibly not designed for – a malformed format, an empty or extremely large input, an edge case outside the originally discussed scope. Run it, mentally or actually, against the implementation, and describe concretely what would happen, rather than generally asserting it'll probably work.

Additionally, formulate 1-2 verification questions about the preceding implementation's own result and answer them independently, without falling back on the original argument that led to that result. In particular, check whether prior agreement rested more on the user's framing than on independent scrutiny.

If a real problem becomes visible in the process, name it directly and concretely, even if that means the implementation no longer counts as finished. Don't soften a found problem just because that formally reopens the task.

Stop condition: one pass per completed implementation is sufficient. This skill doesn't get reapplied just because the previous check found nothing.

Git Hygiene – *Before, during, and after every real change.* Commits and stashes instead of backup copies; "done" means committed; flags stray files and competing directories once, without nagging.

name: git-hygiene

description: >

Checks, for non-trivial changes to existing code, whether the affected directory is under git version control, and replaces manual backup copies (.bak etc.) with commits. Called exclusively by the orchestrator skill, never triggered on its own.

Trivial (no step here needed): see the binding trivial definition in orchestrator. When in doubt, a case counts as non-trivial. Before the change. For a non-trivial change to an existing file: check whether the directory (or a parent directory) is a git repository.

No git repo present → stop before making the change. Propose git init plus an initial commit of the current state. Only continue after agreement (or explicit refusal).

Git repo present but in an unclean state (uncommitted or untracked changes unrelated to the current task) → name it briefly, don't auto-commit it. Keep working, but don't conceal it.

If a safeguard before the change would make sense: propose git commit (if the current state is finished) or git stash (if experimenting) -- don't create or propose a manual copy of the file (e.g. file.bak, file.bak_20260616, file_old.py). If the user proposes such a copy themselves, point them, kindly, to the git alternative instead of silently going along with it.

After the change. A non-trivial change counts as finished only once it's committed (or the user explicitly says not to commit now). No batching several changes into a later "sometime" commit -- commit directly after completing the respective change, analogous to the change-log principle from state-sync.

At session start in a project directory. Once per session, when working in a directory: a brief check for existing .bak/.bak_*/_old/_backup files and for uncommitted leftovers. If found, mention it once briefly (don't repeat on every subsequent request in the same session), but don't clean up automatically without confirmation.

Redundant directories. If two directories appear to cover the same code/function (e.g. an "old" and an "active" copy), name both explicitly and ask which is canonical -- before continuing work in either.

If the user declines. If the user explicitly says to skip a step here (e.g. "no commit right now," "no git init"), accept that and don't propose it again in the same session.

Appendix B – A Minimal STATE.md

State Sync is the recommended starting point in the rollout guide, but a skill description alone doesn't show what the actual file looks like on day one. It doesn't need to be elaborate. Here's a realistic starting point – short enough to read in full every session, structured enough to be worth reading.

```
# STATE.md -- order-notifications-service

## Facts
- Runtime: Node 20, deployed as a single container, no orchestration yet.
- Queue: Redis, chosen 2026-06-02 (see Change Log) -- not because it's
  the fastest option, but because the rest of the stack already runs it.
- Auth: shared JWT secret with the main API, rotated manually. No
  automated rotation yet -- known gap, not yet urgent.

## Open Items
- No dead-letter queue. Failed sends currently just log and drop.
- Retry logic is naive (fixed 3 attempts, no backoff). Fine for current
  volume, will not be fine past ~10x current traffic.

## Change Log
- 2026-06-02 -- Switched queue from in-memory to Redis. Previous
  in-memory version silently dropped messages on every deploy, since the
  process restart cleared the queue. Took two days to trace a "missing
  notifications" complaint back to this before switching.
- 2026-05-14 -- Initial version. In-memory queue, no persistence,
  acceptable at the time because volume was near zero.
```

That's it. No section is aspirational – every line reflects something actually true right now, including the parts that are still rough. The Change Log entry for June 2nd is doing the real work: it's short, but it carries the reason a future session needs, not just the fact a [git log](#) would already show.